Objetivo

Comparar dos formas de recuperar documentos en un corpus de 10 textos cortos con 6 consultas (cada una con un único documento relevante anotado): (1) la línea base léxica, TF-IDF con similitud coseno, y (2) la semántica latente, LSA sobre la misma matriz TF-IDF proyectada a 4 dimensiones con TruncatedSVD. Se evalúan Hit@1, Hit@3 y MRR, se identifica en qué consultas falla cada método mirando los términos que comparten consulta y documento relevante, y se discute por qué un recuperador denso resolvería el caso fallido (q5) y qué costo tiene frente a TF-IDF.

Método

- **Corpus y juicios:** 10 documentos (d01–d10) y 6 consultas (q1–q6), con relevantes d04, d03, d08, d05, d09 y d02 respectivamente.
- **Línea base léxica:** `TfidfVectorizer` de scikit-learn con parámetros por defecto sobre los 10 documentos; el vocabulario resultante tiene 105 términos. Cada consulta se vectoriza con ese mismo vocabulario y se ordenan los 10 documentos por similitud coseno.
- **Semántica latente:** `TruncatedSVD(n_components=4, random_state=0)` sobre la misma matriz TF-IDF; documentos y consultas se comparan por coseno en el espacio latente de 4 dimensiones. Usar la misma matriz TF-IDF de partida hace la comparación justa: ambos métodos difieren solo en el espacio de representación.
- **Métricas:** Hit@1 y Hit@3 (si el relevante queda en el puesto 1 o dentro de los primeros 3) y MRR, la media de 1/rank del relevante sobre las 6 consultas.
- El código (Python con scikit-learn) se entrega aparte; todas las cifras de este reporte provienen de esa ejecución.

Resultados

Tabla 1 — Métricas globales sobre las 6 consultas (Parte 2).

Método	Hit@1	Hit@3	MRR
TF-IDF (coseno)	0.8333	0.8333	0.8611
LSA-4 (TruncatedSVD, coseno)	0.6667	0.8333	0.7500
Diferencia (LSA – TF-IDF)	–0.1667	0.0000	–0.1111

El 0.8333 de TF-IDF son 5 aciertos en el puesto 1 de 6; su MRR de 0.8611 refleja esos cinco aciertos más un 1/6 por q5. En LSA, el Hit@1 baja a 4 de 6 porque q4 cae al puesto 3.

Tabla 2 — Posición del documento relevante por consulta (Parte 3).

Consulta	Relevante	Rank TF-IDF	Rank LSA-4	Falla (fuera del top-3)
q1	d04	1	1	—
q2	d03	1	1	—
q3	d08	1	1	—
q4	d05	1	3	—
q5	d09	6	6	TF-IDF y LSA
q6	d02	1	1	—

Datos de la ejecución que sustentan el análisis: los 4 componentes de la SVD explican 0.0342, 0.1266, 0.1160 y 0.1186 de varianza (0.3953 acumulada). En q5, TF-IDF asigna al relevante d09 una similitud de 0.0790, muy por debajo del primer puesto d03 con 0.3107; en LSA, d09 llega a 0.4999 pero d04 lo supera con 0.9427. En q4, LSA baja a d05 al puesto 3 porque d01 (0.9220) y d10 (0.9206) superan a d05 (0.8984).

Discusión

Dónde falla cada método y por qué (Parte 3). TF-IDF solo falla q5: la consulta ("más determinista al elegir la siguiente palabra") y d09 ("la temperatura reescala los logits… valores bajos concentran la probabilidad") no comparten ningún término de contenido; solo coinciden palabras funcionales como "la", "el/del" y "los". Como

el vectorizador por defecto no elimina stopwords en español, esos términos genéricos dominan la similitud, y d03 —que sí comparte "modelo", "la", "al" y "los" con la consulta— gana con 0.3107 frente al 0.0790 de d09. LSA tampoco resuelve q5 (d09 sigue en el puesto 6) y además degrada q4 del puesto 1 al 3: con solo 4 componentes que capturan 0.3953 de la varianza, el espacio latente mezcla temas y documentos generales como d01 y d10 se acercan demasiado a la consulta. Con 10 documentos, la SVD no tiene datos para aprender una estructura latente útil; de ahí que su Hit@1 caiga 0.1667 y su MRR 0.1111 respecto a TF-IDF, sin ganar nada en Hit@3.

Por qué un recuperador denso resolvería q5 (Parte 4). Un codificador entrenado (embeddings de un modelo preentrenado con pares pregunta-pasaje) no depende de coincidencias léxicas: aprende de grandes corpus que "elegir la siguiente palabra de forma más determinista" y "valores bajos de temperatura concentran la probabilidad" describen el mismo concepto, y los proyecta cerca en el espacio vectorial. Eso ataca justo el desajuste de vocabulario que hace fracasar a TF-IDF y a LSA-4 en q5, donde no hay ningún término de contenido compartido que puentear. El costo frente a TF-IDF es triple: (1) exige un modelo preentrenado, con su memoria y su cómputo de inferencia (en la práctica, GPU) para codificar cada consulta y todos los documentos del índice; (2) requiere entrenamiento previo y puede degradarse fuera de su dominio, mientras que TF-IDF no entrena nada y funciona igual en cualquier colección; y (3) pierde interpretabilidad: con TF-IDF se puede señalar qué términos coinciden (aquí, ninguno de contenido, y por eso el error es diagnosticable), mientras que en un embedding denso la cercanía no es auditable término a término. En un corpus de 10 documentos ese costo no se justifica, pero q5 muestra el techo de los métodos no entrenados: sin señal semántica externa, ni el léxico puro ni una SVD de rango 4 sobre 10 documentos pueden conectar "determinista" con "temperatura".